Menghidupkan Bahasa di Era AI

Kisah, Sains, dan Rekayasa di Balik Mesin Penerjemah Sangiang

Membangun Jembatan Digital untuk Bahasa Daerah Nusantara

Inisiator Proyek:
Yayasan Cipta Mandala Dwipa

Rekayasa Sistem AI dan Penerjemah:
Divisi DigiLab CMD

03 November 2026

Menghidupkan Bahasa di Era AI:
Kisah, Sains, dan Rekayasa di Balik Mesin Penerjemah Bahasa Sangihe (Sangiang)

Inisiator Proyek:
Yayasan Cipta Mandala Dwipa

Rekayasa Sistem AI dan Penerjemah:
Divisi DigiLab CMD

Pernyataan Kepemilikan Hak Cipta:

- **Yayasan Cipta Mandala Dwipa:** Pemilik hak cipta korpus terjemahan *Kamus Sangihe-Indonesia*. Hak Cipta © 2026 dilindungi oleh undang-undang.
- **Divisi DigiLab CMD:** Tim rekayasa sistem kecerdasan buatan (AI) dan penerjemah korpus.
- **Brill / KITLV (Koninklijk Instituut voor Taal-, Land- en Volkenkunde):** Pemilik hak cipta buku-buku kamus terkait, khususnya karya leksikografi klasik *Sangihe-Nederlandsch Woordenboek* (1959) oleh K.G.F. Steller & W.E. Aebersold.
- **Lembaga Alkitab Indonesia (LAI):** Pemilik hak cipta buku-buku teks Kitab Suci terkait, mencakup Alkitab Terjemahan Bahasa Sangihe, Terjemahan Baru (TB), dan Alkitab Bahasa Indonesia Masa Kini (BIMK).

Misi & Dedikasi:
Buku dan sistem teknologi ini dipersembahkan untuk masyarakat penutur Bahasa Sangihe di Kepulauan Sangihe, Sulawesi Utara, perantauan Nusa Utara, serta seluruh pecinta kebudayaan dan bahasa daerah Nusantara.

Tipografi & Kompilasi Teknis:
Buku ini disusun dalam format LaTeX dan dikompilasi menggunakan mesin XeLaTeX.
Menggunakan jenis huruf *Gentium Plus* karya SIL International untuk memastikan seluruh glif fonemis dan tanda diakritik ortografi Bahasa Sangihe (ạ, ẹ, ḛ, ĕ, ị, ọ, ụ, serta *ḷ*) tercetak dengan sempurna.

Cetakan Digital: Edisi Pertama – 03 November 2026

Pengantar untuk Pembaca

Buku ini ditulis bukan hanya untuk para peneliti kecerdasan buatan, melainkan untuk siapa saja yang mencintai bahasa daerah, kebudayaan Nusantara, dan ingin memahami bagaimana teknologi modern seperti *Large Language Model* (LLM) dapat digunakan untuk menyelamatkan dan melestarikan warisan leluhur kita.

Proyek ini terwujud berkat inisiasi dan visi mulia dari **Yayasan Cipta Mandala Dwipa** yang berkomitmen pada pelestarian kebudayaan serta kemajuan peradaban literasi digital bahasa daerah. Setiap konsep teknis yang tampak rumit—mulai dari kecerdasan buatan, basis data, hingga struktur morfologi kata—akan dijelaskan dengan bahasa yang lugas, analogi sehari-hari, dan ilustrasi yang mudah dibayangkan.

Contents

I Mengenal Bahasa Sangihe dan Tantangan Digitalnya **1**

1 Menghidupkan Kembali Warisan Bahasa di Era Digital **3**
1.1 Bahasa Sangihe: Mutiara Kebudayaan di Tapal Batas Utara . . . 3
1.2 Tantangan AI: Mengapa Bahasa Daerah Kerap Tertinggal? . . . 3
1.3 Misi Proyek: Membangun Mesin Penerjemah Cerdas . . . 4

2 Seni Bunyi dan Rahasia Tanda Diakritik Sangihe **5**
2.1 Mengapa Tanda Kecil Membawa Arti Besar? . . . 5
2.2 Dilema Papan Ketik Modern: Masalah Mengetik di Ponsel . . . 5
2.3 Strategi Dua Jalur: Penormalan dan Pemulihan Diakritik . . . 6

3 Morfologi Sangihe: Seni Membongkar dan Menyusun Kata **7**
3.1 Morfologi: Mengapa Kata Dasar Bisa Berubah Wujud? . . . 7
3.2 Keunikan Sisipan di Dalam Perut Kata . . . 7
3.3 Hukum Alih Bunyi: Studi Kasus Nyata Kata *Waes* . . . 8

II Mengumpulkan Harta Karun Kata: Korpus & Kamus **9**

4 Mengumpulkan Harta Karun Kata: Korpus & Kamus Klasik **11**
4.1 Bahan Bakar Utama Kecerdasan Buatan: Data Paralel . . . 11
4.2 Lima Sumber Daya Utama Proyek Ini . . . 11
4.2.1 1. Kamus Klasik Steller-Aebersold (11,391 Lema) . . . 11
4.2.2 2. Teks Paralel Kitab Suci Sangihe (29,671 Ayat) . . . 12
4.2.3 3. Sastra Klasik: Dongeng Sasahara & Wiracarita Himbawo . . . 12
4.2.4 4. Sastra Folklor Daerah: *Tiga Kulano dari Dagho* (407 Pasangan Kalimat) . . . 12
4.2.5 5. Korpus Percakapan Sehari-Hari (54 Pasangan Kalimat) . . . 13
4.3 Totalitas Bank Memori Paralel . . . 13

5 Membersihkan Butiran Emas: Normalisasi dan Pre-processing **15**
5.1 Mengapa Teks Mentah Tidak Boleh Langsung Diproses? . . . 15
5.2 Langkah-Langkah Pembersihan Teks . . . 15

6 Ujian yang Jujur: Tolok Ukur Benchmark & Anti-Kebocoran Data **17**
6.1 Mengapa Kita Butuh Ujian yang Independen? . . . 17
6.2 Menyusun 150 Kalimat Uji Acuan . . . 17
6.3 Prinsip Ketat Anti-Kebocoran Data (*Anti-Leakage*) . . . 18
6.4 Tiga Rapor Penilaian Kualitas . . . 18

III Arsitektur Cerdas: Teknologi Hybrid RAG **19**

7 Arsitektur Cerdas: Mengapa Memilih Teknologi Hybrid RAG? **21**
7.1 Dua Cara Belajar: Menghafal Mati vs Ujian Buka Buku 21
7.2 Mengapa Harus *Hybrid* (Campuran Dua Tingkat)? 21

8 Mesin Pencari Secepat Kilat: Menyelamatkan Komputer dari Kemacetan Memori **23**
8.1 Tantangan Nyata di Komputer Pribadi 23
8.2 Kisah Kemacetan Memori: Tragedi Model yang Terus Tergusur 23
8.3 Solusi Elegan: Beralih ke Mesin Pencari SQLite BM25 24

9 Memandu Sang Model: Seni Rekayasa Prompt & Pemilihan Nomor Arti **25**
9.1 Prompt: Mengajar Asisten Pintar Cara Bekerja 25
9.2 Tantangan Kata Bermakna Ganda (Polisemi) 25
9.3 Solusi: Panduan Disambiguasi Leksikal Otomatis 25
9.4 Sentuhan Akhir: Pemulihan Diakritik Otomatis 26

IV Melatih Otak AI: Seni Fine-Tuning Model **27**

10 Melatih Otak AI: Mengapa dan Bagaimana Fine-Tuning Bekerja? **29**
10.1 Dari Dokter Umum Menjadi Dokter Spesialis 29
10.2 Mengapa Tidak Melatih dari Nol (*Train from Scratch*)? 29
10.3 Keajaiban Teknologi LoRA dan QLoRA 4-bit 29

11 Melatih Saraf di Cloud: Membaca Rapor Belajar dan Kurva Loss **31**
11.1 Di Balik Layar Ruang Kelas AI 31
11.2 Memahami Konsep *Loss*: Rapor Tingkat Kesalahan AI 31
11.3 Data Empiris Nyata: Grafik Kemajuan Belajar 31
11.4 Analisis Bebas *Overfitting*: Tanda Keberhasilan Sejati 33
11.5 Evolusi Sangiang v2: Mengatasi Kendala Teknis & Memperluas Ranah Multi-Task . . . 33

12 Membawa Pulang Sang Juara: Kuantisasi GGUF & Ollama Lokal **35**
12.1 Dari Awan Komputasi ke Komputer di Meja Kerja 35
12.2 Seni Kuantisasi Q4_K_M: Mengompresi Tanpa Mengurangi Kecerdasan 35
12.3 Pendaftaran Otomatis ke Ollama Lokal 36

V Menikmati Hasil Karya: Aplikasi & Uji Kualitas **37**

13 Jembatan Penghubung: Arsitektur Layanan REST API **39**
13.1 Apa Itu API dan Mengapa Kita Membutuhkannya? 39
13.2 Membangun dengan FastAPI: Cepat, Ramping, dan Modern 39

14 Ruang Kerja Interaktif: Antarmuka Web Translation Workbench **41**
14.1 Dari Kode Hitam-Putih Menjadi Pengalaman Visual yang Memikat 41
14.2 Fitur-Fitur Unggulan Meja Kerja Digital 41
14.3 Antarmuka Percakapan Cerdas: Sangiang Chatbot (Port 2026) 42

15 Menguji Keunggulan: Evaluasi Komparatif Model Base vs Fine-Tuned **43**
15.1 Pertarungan Dua Generasi Model 43
15.2 Tiga Temuan Kunci dari Pengujian 43
15.3 Menatap Masa Depan: Langkah Ke Depan Pelestarian Bahasa 44

Lampiran: Ringkasan Kaidah Morfologi Sangihe **45**

Part I

Mengenal Bahasa Sangihe dan Tantangan Digitalnya

Chapter 1

Menghidupkan Kembali Warisan Bahasa di Era Digital

1.1 Bahasa Sangihe: Mutiara Kebudayaan di Tapal Batas Utara

Di ujung utara kepulauan Nusantara, terbentang gugusan pulau yang indah di wilayah Nusa Utara (Kabupaten Kepulauan Sangihe, Sulawesi Utara). Di tanah berombak inilah Bahasa Sangihe (*Bahasa Sangir*) lahir, tumbuh, dan dituturkan secara turun-temurun oleh para pelaut ulung, penutur sastra lisan, dan masyarakat adat. Bahasa ini sarat dengan kearifan lokal, filosofi kehidupan bahari, serta karya sastra bernilai tinggi seperti wiracarita *Sasahara* dan puisi epik *Biong i Himbawo*.

Namun, seiring derasnya arus modernisasi dan peralihan generasi penutur muda ke bahasa pergaulan umum (Melayu Manado dan Bahasa Indonesia gaul), keberadaan bahasa daerah ini menghadapi tantangan yang nyata. Jika tidak ada upaya serius untuk mendokumentasikannya ke dalam bentuk digital dan menghadirkannya di gawai modern, bahasa yang kaya ini berisiko perlahan-lahan memudar dari percakapan sehari-hari.

1.2 Tantangan AI: Mengapa Bahasa Daerah Kerap Tertinggal?

Dalam beberapa tahun terakhir, dunia menyaksikan lompatan raksasa dalam teknologi kecerdasan buatan (*Artificial Intelligence* / AI). Model-model bahasa besar (*Large Language Model* atau LLM) seperti ChatGPT atau Qwen mampu menjawab pertanyaan rumit, menulis esai, bahkan menerjemahkan berbagai bahasa dunia dengan fasih.

Catatan Arsitektur: Analogi Perpustakaan Raksasa Dunia

Bayangkan sebuah model AI modern seperti seorang sarjana pintar yang telah membaca miliaran buku dari perpustakaan terbesar dunia. Masalahnya, 95% buku di perpustakaan tersebut ditulis dalam Bahasa Inggris, Mandarin, Spanyol, atau Bahasa Indonesia baku. Bahasa-bahasa daerah seperti Bahasa Sangihe hampir tidak pernah ia temukan di rak perpustakaan itu. Akibatnya, saat kita mengajaknya berbicara dalam Bahasa Sangihe, sang sarjana akan bingung atau sekadar menebak-nebak kata.

Dalam bidang linguistik komputasi, bahasa seperti Sangihe dikategorikan sebagai *low-resource language* (bahasa berlindung sumber daya komputasi minim). Tantangan utamanya meliputi:

1. **Kelangkaan Data Digital:** Tidak banyak situs web atau artikel daring yang memuat teks bahasa Sangihe dalam jumlah besar.

2. **Kerumitan Tanda Diakritik:** Huruf vokal dalam bahasa Sangihe memiliki penanda khusus yang menentukan arti, yang kerap hilang saat diketik di ponsel pintar.
3. **Morfologi yang Kaya:** Satu kata dasar bisa berubah wujud menjadi puluhan bentuk baru melalui awalan, sisipan di tengah kata, dan akhiran.

1.3 Misi Proyek: Membangun Mesin Penerjemah Cerdas

Proyek ini lahir dengan sebuah cita-cita mulia namun terukur: **Membangun mesin penerjemah dua arah antara Bahasa Sangihe dan Bahasa Indonesia yang akurat, menghormati kaidah bahasa formal, dan dapat dijalankan di komputer biasa secara luring (offline) tanpa bergantung pada peladen cloud berbayar.**

Untuk mewujudkannya, proyek ini memadukan dua pendekatan teknologi mutakhir:

- **Teknologi Hybrid RAG (*Retrieval-Augmented Generation*):** Membekali sang model AI dengan ”buku pintar” berupa kamus digital leksikon klasik dan bank ingatan kalimat paralel yang dapat ia buka setiap kali diminta menerjemahkan.
- **Pelatihan Khusus (*Fine-Tuning*):** Melatih kembali saraf pemikiran model fondasi `Qwen2.5-7B-Instruct` dengan puluhan ribu contoh kalimat terjemahan nyata agar ia memahami gaya, tata krama, dan jiwa Bahasa Sangihe.

Buku ini akan mengajak Anda menyusuri perjalanan lengkap bagaimana teknologi ini dirancang dari nol, langkah demi langkah, dengan penjelasan yang bersahabat bagi siapa saja.

Chapter 2

Seni Bunyi dan Rahasia Tanda Diakritik Sangihe

2.1 Mengapa Tanda Kecil Membawa Arti Besar?

Dalam Bahasa Indonesia, kata *tahu* (mengerti) dan *tahu* (makanan berbahan kedelai) ditulis persis sama tanpa tanda baca khusus. Kita hanya bisa membedakannya lewat konteks kalimat. Namun, dalam Bahasa Sangihe, perbedaan bunyi vokal memiliki pengaruh yang sangat mendalam dan mutlak dalam menentukan arti suatu kata.

Untuk mengabadikan perbedaan bunyi tersebut ke dalam tulisan, para linguis dan penerjemah klasik (termasuk Dr. N. Adriani, K.G.F. Steller, dan W.E. Aebersold) merumuskan sistem ejaan dengan **tanda diakritik** (*penanda aksen di atas atau di bawah huruf*).

Kaidah Linguistik Sangihe: Daftar Bunyi Khusus Bahasa Sangihe

Bahasa Sangihe mengenal bunyi-bunyi khas yang tidak dimiliki oleh Bahasa Indonesia standar:

- **Vokal Bertitik Bawah (ạ, ẹ, ị, ọ, ụ):** Menandakan vokal tertutup atau vokal panjang yang diucapkan dengan bibir lebih terkatup atau posisi lidah yang lebih tinggi.
- **Pepet (ĕ):** Huruf e lemah seperti bunyi huruf e pada kata Indonesia *sedap* atau *segar*.
- **Kombinasi Bertitik-Pepet (ḛ):** Bunyi vokal yang menggabungkan artikulasi tertutup dan nada sentak.
- **Lateral Terpalatisasi ($l^{\sim}$ / l̪):** Ditulis dengan huruf l yang diberi tanda gelombang kecil di bawahnya (Unicode U+1DCA). Diucapkan seperti huruf l yang lidahnya melengkung lembut ke langit-langit mulut.

2.2 Dilema Papan Ketik Modern: Masalah Mengetik di Ponsel

Di zaman modern saat ini, hampir semua orang berkomunikasi menggunakan papan ketik standar (QWERTY) di ponsel pintar atau laptop. Di papan ketik tersebut, tidak ada tombol untuk huruf **ạ**, **ĕ**, atau **l̪**.

Catatan Arsitektur: Analogi Nada Musik Lagu Daerah

Bayangkan sebuah partitur lagu daerah yang indah. Nada aslinya memiliki tanda nada tinggi, nada rendah, dan tanda jeda istirahat. Namun, saat dinyanyikan di jalanan tanpa teks, orang-orang hanya mengingat lirik dasarnya saja tanpa tanda not balok. Tugas mesin penerjemah pintar kita adalah seperti seorang guru paduan suara yang andal: meskipun pendengar hanya bersenandung datar (mengetik tanpa diakritik), sang guru tetap tahu nada yang benar dan mampu menuliskan kembali not balok lengkap dengan tanda lagunya secara sempurna.

Oleh karena itu, salah satu aturan emas (*core requirement*) yang ditetapkan sejak awal pembuatan sistem ini adalah:

"Input tanpa tanda diakritik harus diterima dengan lapang dada, namun keluaran hasil terjemahan Bahasa Sangihe harus tetap mengembalikan tanda diakritik yang lengkap dan akurat."

2.3 Strategi Dua Jalur: Penormalan dan Pemulihan Diakritik

Bagaimana sistem komputer kita mengatasi dilema ini? Kami merancang teknik yang disebut **Normalisasi Fonemis Dwiarah**:

1. **Jalur Masuk (Penormalan Kata):** Saat pengguna mengetik kata `mebaese su waesu` (tanpa tanda apa pun), modul `normalizer.py` akan membersihkan huruf-huruf tersebut menjadi bentuk dasar. Huruf bertitik dan bertanda disetarakan ke huruf alfabet biasa (a, e, i, o, u). Tujuannya agar mesin pencari kamus dapat mencocokkannya dengan cepat di database leksikon.
2. **Jalur Keluar (Pemulihan Diakritik):** Setelah model AI menerjemahkan kalimat, modul cerdas `DiacriticRestorer` akan memeriksa glosarium terverifikasi dan teks rujukan paralel. Modul ini secara otomatis menyisipkan kembali tanda diakritik yang sah, sehingga kata berubah kembali menjadi ejaan formal: ***mĕbaesę̌ su waes-u***.

Dengan rancangan ini, pengguna yang awam tidak perlu repot mencari simbol tersembunyi di papan ketik saat mengetik kalimat, namun hasil terjemahan yang dihasilkan tetap menjunjung tinggi kehormatan ortografi asli Bahasa Sangihe.

Chapter 3

Morfologi Sangihe: Seni Membongkar dan Menyusun Kata

3.1 Morfologi: Mengapa Kata Dasar Bisa Berubah Wujud?

Dalam ilmu bahasa, **morfologi** adalah studi tentang bagaimana kata dibentuk dari potongan-potongan makna terkecil. Jika dalam Bahasa Indonesia kita mengenal kata dasar *ajar* yang bisa menjadi *belajar, mengajar, terpelajar*, maka dalam Bahasa Sangihe variasinya jauh lebih kaya dan dinamis.

Catatan Arsitektur: Analogi Bongkar Pasang Pakaian

Bayangkan sebuah kata dasar seperti sesosok tubuh manusia. Morfologi adalah seni memakaikan pakaian pada tubuh tersebut:

- Memakaikan topi di kepala disebut **Awalan (Prefiks)**, seperti *mĕ-*, *ma-*, atau *ipang-*.
- Memakaikan sabuk di tengah pinggang disebut **Sisipan (Infiks)**, seperti menyisipkan suku kata *-um-* atau *-in-* tepat di tengah perut kata!
- Memakaikan sepatu di kaki disebut **Akhiran (Sufiks)**, seperti *-ang*, *-eng*, atau *-e*.
- Mengajak bayangannya ikut berjalan bersama disebut **Pengulangan (Reduplikasi)**.

Tugas modul komputer kita (`stemmer.py`) adalah seperti seorang detektif yang melihat seseorang berpakaian lengkap, lalu menanggalkan topinya, melepas sabuknya, dan mencopot sepatunya agar dapat mengenali siapa sosok orang (kata dasar) yang sebenarnya!

3.2 Keunikan Sisipan di Dalam Perut Kata

Salah satu ciri khas rumpun bahasa Filipina dan Austronesia utara yang diwarisi oleh Bahasa Sangihe adalah keberadaan **infiks** (sisipan). Di sebagian besar bahasa dunia, imbuhan hanya dipasang di depan atau di belakang kata. Namun di Bahasa Sangihe, imbuhan diselipkan ke sela-sela huruf pertama dan kedua kata dasar:

Kaidah Linguistik Sangihe: Contoh Sisipan -um- dan -in-

Perhatikan bagaimana kata dasar bertransformasi saat disisipi makna tindakan atau kala lampau:

- Kata dasar: **tampa** (*tempat / tiba*) $\rightarrow$ Disisipi *-um-* di antara t dan a $\rightarrow$ **t-um-ampa** (*sedang tiba / datang melangkah*).
- Kata dasar: **koro** (*bakar*) $\rightarrow$ Disisipi *-in-* $\rightarrow$ **k-in-oro** (*telah dibakar*).

Jika modul pencari di komputer tidak memahami aturan sisipan ini, komputer akan mencari kata *tumampa* di kamus dan tidak akan pernah menemukannya, karena kata tersebut terdaftar di bawah huruf T (*tampa*), bukan huruf T-U!

3.3 Hukum Alih Bunyi: Studi Kasus Nyata Kata *Waes*

Dalam perjalanan pengembangan translation engine ini, kami menemukan sebuah kasus menarik yang membuktikan betapa pentingnya pemahaman morfologi yang mendalam.

Dalam dongeng klasik Sangihe, terdapat sebuah kalimat terkenal:

*"Mahi iąko mĕbaesẹ̆ su **waes-u**."*

Awalnya, sistem penerjemah menerjemahkan kata **waes-u** dengan arti "kemarau-mu", padahal arti yang tepat dalam konteks cerita binatang tersebut adalah **"sarang-mu"**! Mengapa kebingungan ini bisa terjadi?

1. Kata aslinya berbunyi **baesẹ̆** (lema nomor 331 di kamus Steller-Aebersold).
2. Saat kata benda ini diberi kata ganti milik *-u* ("milikmu"), bahasa Sangihe memberlakukan hukum fonemis:
   - Konsonan bibir b bermutasi melunak menjadi w ($b \leftrightarrow w$).
   - Huruf vokal gema (*echo vowel*) di ujung kata luruh, menyisakan akar *waes*.
   - Ditambahkan akhiran *-u*, menjadi **waes-u**.

Dengan menanamkan aturan alih bunyi ini ke dalam berkas `morphology/stemmer.py`, komputer kini secara otomatis tahu bahwa kata *waes* berasal dari lema asli ***baesẹ̆***. Mesin pencari kamus pun berhasil menemukan arti lema yang sebenarnya di dalam database hanya dalam waktu kurang dari 2 milidetik!

Part II

Mengumpulkan Harta Karun Kata: Korpus & Kamus

Chapter 4

Mengumpulkan Harta Karun Kata: Korpus & Kamus Klasik

4.1 Bahan Bakar Utama Kecerdasan Buatan: Data Paralel

Sebuah mobil secanggih apa pun tidak akan dapat melaju tanpa bahan bakar. Demikian pula dengan kecerdasan buatan: model AI secanggih apa pun tidak akan bisa menerjemahkan bahasa tanpa adanya data contoh kalimat yang benar.

Dalam dunia penerjemahan mesin, data paling berharga disebut **Korpus Paralel** (*Parallel Corpus*).

Catatan Arsitektur: Analogi Buku Dua Bahasa dan Teks Film

Bayangkan sebuah buku cerita yang di halaman kirinya dicetak dalam Bahasa Sangihe, dan tepat di halaman kanannya dicetak terjemahan resminya dalam Bahasa Indonesia kalimat demi kalimat. Atau bayangkan sebuah film yang menampilkan dua baris terjemahan teks (subtitle) sekaligus di layar. Dengan membandingkan ribuan pasangan kalimat tersebut, kecerdasan buatan dapat mempelajari bagaimana frasa tertentu dialihbahasakan secara alami.

4.2 Lima Sumber Daya Utama Proyek Ini

Proyek ini mengintegrasikan lima warisan dokumen besar yang telah didigitisasi:

4.2.1 1. Kamus Klasik Steller-Aebersold (11,391 Lema)

Kamus monumental *Sangihe-Nederlandsch Woordenboek* karya K.G.F. Steller dan W.E. Aebersold (1959) diterbitkan di bawah naungan Koninklijk Instituut voor Taal-, Land- en Volkenkunde (KITLV), dengan hak cipta buku dipegang oleh **Brill / KITLV**.

Adapun korpus terjemahan kamus tersebut ke dalam Bahasa Indonesia—yang dikerjakan secara cermat dan komprehensif oleh tim penerjemah **Divisi DigiLab CMD** dan menjadi basis leksikal komputasi dalam proyek ini—merupakan hak cipta milik **Yayasan Cipta Mandala Dwipa**. Di dalam sistem, data ini dimuat dari basis data SQLite `sarindi.sqlite` yang mencakup:

- **11,391 lema utama** lengkap dengan tanda diakritik fonemis dan penjelasan arti rinci.
- **Indeks Balik Bahasa Indonesia (*id_index*):** Memungkinkan kita mencari padanan kata Sangihe dari sudut pandang kosakata Bahasa Indonesia.

- **Pencarian Teks Cepat (SQLite FTS5):** Memungkinkan komputer menemukan lema yang cocok dalam hitungan sepersekian milidetik.

4.2.2 2. Teks Paralel Kitab Suci Sangihe (29,671 Ayat)

Naskah terjemahan Alkitab Bahasa Sangihe yang disandingkan berdampingan dengan Alkitab Terjemahan Baru (TB) dan Bahasa Indonesia Masa Kini (BIMK). Hak cipta dari seluruh naskah dan buku teks Kitab Suci ini dipegang oleh **Lembaga Alkitab Indonesia (LAI)**. Sumber ini menyumbang lebih dari 29 ribu pasangan ayat dengan register bahasa formal, tata kalimat yang tertib, dan diakritik yang terjaga rapi.

4.2.3 3. Sastra Klasik: Dongeng Sasahara & Wiracarita Himbawo

Untuk melengkapi ragam bahasa percakapan, metafora bahari, dan folklor tradisional, proyek ini memasukkan:

- **27 Bab Dongeng Klasik Sangihe (28 Kalimat):** Kisah binatang, fabel moral, dan legenda rakyat (seperti kisah Gurita dan Ular Laut).
- **Puisi Epik *Biong i Himbawo* (136 Larik):** Nyanyian kepahlawanan dengan struktur persajakan puitis tingkat tinggi.

4.2.4 4. Sastra Folklor Daerah: *Tiga Kulano dari Dagho* (407 Pasangan Kalimat)

Sebagai terobosan mutakhir untuk memutus ketergantungan model terhadap domain religius (bias 99% teks Alkitab), proyek ini mengintegrasikan buku kumpulan cerita rakyat dwibahasa *Tiga Kulano dari Dagho* (Paul Nebath dkk., Departemen Pendidikan dan Kebudayaan, 1983). Korpus ini mencakup 14 bab kisah kepahlawanan, fabel satwa, mitologi laut, dan asal-usul tanjung yang tersebar di wilayah kepulauan:

- **10 Cerita Sangihe Besar (286 Pasangan):** Termasuk wiracarita *Gumansalangi*, putri legendaris *Tomatiti*, pahlawan bahari *Tiga Orang Pemberani dari Dagho*, *Ompung* (dewa lautan), serta fabel satwa jenaka *Kera dan Bangau*.
- **2 Cerita Bahasa Talaud (70 Pasangan):** Kisah manusia pertama di Pulau Karakelang (*Alamona Ntaumata Ntaloda*) dan hikmah anak yang durhaka (*Battiu Anao*).
- **1 Cerita Dialek Tagulandang (32 Pasangan):** Legenda terbentuknya tanjung terputus (*Tonggeng Napoto*).
- **1 Cerita Dialek Siau (19 Pasangan):** Fabel musyawarah satwa *Kina Hetung dingangu Manu Banggiung*.

Catatan Arsitektur: Pemulihan Filologis & Rekonstruksi Halaman Tertukar

Pada buku cetakan fisik tahun 1983, keterbatasan teknologi cetak saat itu memaksa penyunting menambahkan tanda-tanda diakritik menggunakan coretan pena manual di atas ketikan mesin tik, yang saat dipindai melalui OCR menghasilkan ribuan derau karakter. Selain itu, ditemukan anomali perwajahan kritis di mana potongan naskah Bahasa Sangihe antara Cerita 7 (*Walawo, Urang, Lipang...*) dan Cerita 12 (*Datu I Ralero*) tertukar halamannya pada lembaran cetak.
Melalui pipeline restorasi berbasis leksikon Steller-Aebersold (`scripts/process_tiga_kulano.py`) dan algoritma *Dynamic Programming Sentence Alignment*, kedua naskah berhasil disambung kembali sesuai koherensi plot aslinya, serta dibersihkan ke dalam ortografi fonemis baku lengkap dengan konsonan kepak retrofleks (*ḷ*) dan vokal pepet (*ĕ, ĕ́*).

4.2.5 5. Korpus Percakapan Sehari-Hari (54 Pasangan Kalimat)

Untuk menjembatani model dengan situasi percakapan nyata di dunia modern, proyek ini menyaring dan menyelaraskan data kosakata percakapan dari sumber-sumber web terbuka (`data/web/`) serta ungkapan pragmatis kesantunan masyarakat kontemporer. Korpus ini dikurasi secara cermat melalui skrip `scripts/process_web_conversations.py`:

- **Salam Waktu & Sapaan:** Ungkapan salam formal dan santai seperti *Salamate měhěbị* (selamat pagi), *Salamate ěllong darua* (selamat siang), *Salamate sělę* (selamat sore), dan *Salamate hěbi* (selamat malam).
- **Pragmatik Kesantunan:** Frasa penting dalam interaksi sosial seperti *Tarima kasehę* (terima kasih), *Makiangsingkatę* (terima kasih banyak), *Kapurane ią* (maafkan saya), *Tabea* (permisi), dan *Pariurung* (silakan).
- **Tanya Jawab Percakapan:** Pola tanya jawab lazim seperti kabar (*Kere habarę?*), asal (*Bọu apa i kau?*), tujuan (*Sarang apa i kau?*), nama (*Isai arengu?*), serta jamuan makan (*Ndaị e kumâng!*).
- **Panggilan Budaya:** Panggilan kehormatan dan kasih sayang khas suku Sangihe seperti *Ungke / Opo* (pemuda) dan *Wawu* (pemudi).

Kehadiran korpus percakapan ini memberikan penahan (*pragmatic anchor*) yang esensial agar bot percakapan Sangiang (Port 2026) dapat merespons tegur sapa warga dengan luwes, santun, dan alami tanpa mengeluarkan respons yang melantur.

4.3 Totalitas Bank Memori Paralel

Melalui modul pengindeks cerdas (`retrieval/indexer.py`), seluruh data ini dilebur dan disimpan dalam satu basis data terpadu `parallel_corpus.sqlite`. Kini sistem memiliki total **30,296 pasangan kalimat rujukan** yang siap dipanggil kapan saja oleh mesin penerjemah.

Chapter 5

Membersihkan Butiran Emas: Normalisasi dan Pre-processing

5.1 Mengapa Teks Mentah Tidak Boleh Langsung Diproses?

Dalam pepatah ilmu data, dikenal prinsip terkenal: *"Garbage In, Garbage Out"* (jika data yang dimasukkan berantakan, maka hasil yang dikeluarkan komputer juga akan kacau). Sebelum sebuah kamus atau teks kalimat dapat dipakai oleh kecerdasan buatan, teks tersebut harus melalui proses pembersihan dan perapian (*pre-processing*).

Catatan Arsitektur: Analogi Menyaring Pasir Emas

Menambang data dari buku-buku tua mirip dengan mendulang pasir emas di sungai. Di dalam pasir tersebut, terdapat butiran emas berharga (kata-kata bermakna), tetapi juga terdapat kerikil, lumpur, dan dedaunan (tanda kutip ganda yang miring, spasi berlebih, huruf besar-kecil yang tidak konsisten). Modul pre-processing kita bertindak sebagai ayakan halus yang mencuci bersih data mentah sehingga yang tersisa hanyalah kosakata murni yang siap diolah.

5.2 Langkah-Langkah Pembersihan Teks

Modul `morphology/normalizer.py` menjalankan beberapa tugas penting:

1. **Penyatuan Tanda Petik dan Glotal (*Apostrophe Standardization*):** Dalam naskah ketikan, tanda petik sering bervariasi antara tanda petik tegak (`'`), petik miring (`’`), atau aksen (`` ` ``). Modul kami menyatukan semuanya ke satu standar agar pencarian tidak meleset.

2. **Penormalan Spasi & Tanda Baca:** Menghilangkan spasi ganda, merapikan tanda titik-koma, serta memisahkan tanda baca yang menempel pada kata tanpa merusak arti.

3. **Penyaringan Kata Tugas (*Stopwords Filtering*):** Dalam percakapan, kata-kata sambung pendek seperti *su* (di/pada), *u* (dari/milik), *i* (si/penanda nama orang), dan *kụ* (dan/lalu) muncul ribuan kali. Jika komputer mencari arti kata *su* di kamus setiap kali menerjemahkan, proses akan menjadi sangat lambat tanpa memberikan nilai tambah. Sistem kami secara cerdas memfilter kata-kata tugas umum ini saat mencari leksikon utama.

Dengan langkah-langkah di atas, mesin penerjemah dapat berkonsentrasi penuh pada kata-kata kunci pembawa pesan utama dalam kalimat.

Chapter 6

Ujian yang Jujur: Tolok Ukur Benchmark & Anti-Kebocoran Data

6.1 Mengapa Kita Butuh Ujian yang Independen?

Bagaimana kita tahu bahwa sistem penerjemah AI kita benar-benar pandai menerjemahkan, dan bukan sekadar ”menghafal mati” kalimat-kalimat yang pernah ia lihat?

Catatan Arsitektur: Analogi Soal Ujian Akhir Sekolah

Bayangkan seorang guru yang sedang mendidik muridnya menjelang ujian nasional. Selama berbulan-bulan di kelas, sang guru memberikan ratusan latihan soal (ini adalah **Data Latih / Training Set**). Namun, saat hari ujian tiba, sang guru harus mengunci lembar soal ujian akhir di laci tertutup dan tidak boleh membocorkannya kepada murid (ini adalah **Data Uji / Test Benchmark**).
Jika seorang murid mendapatkan nilai 100 karena sebelumnya sudah melihat kunci jawaban soal ujian, nilai tersebut adalah nilai palsu (*data leakage*). Tetapi jika murid tersebut mampu menjawab soal-soal baru yang belum pernah ia lihat sama sekali, barulah kita yakin bahwa ia benar-benar cerdas!

6.2 Menyusun 150 Kalimat Uji Acuan

Untuk menguji kemampuan sistem secara objektif, modul `benchmark/generate_testset.py` menyaring **150 kalimat acuan** pilihan dari seluruh korpus data kita. Kalimat-kalimat ini dipilih dengan metode cuplikan berlapis (*stratified sampling*) yang mencakup:

- Kalimat pendek (sapaan dan ujaran ringkas).
- Kalimat panjang dengan anak kalimat majemuk.
- Kalimat dengan kepadatan tanda diakritik tinggi (ą, ę, ě, ḷ).
- Representasi berimbang dari ketiga domain (teks Alkitab, fabel dongeng rakyat, dan puitika Himbawo).

Seluruh 150 kalimat ini disimpan secara terisolasi di dalam berkas `benchmark/testset.json`.

6.3 Prinsip Ketat Anti-Kebocoran Data (*Anti-Leakage*)

Saat modul `training/prepare_dataset.py` mengekstrak data untuk melatih model bahasa, sistem secara otomatis membaca seluruh identitas unik (ID) dari ke-150 kalimat uji tersebut dan **mengecualikannya secara mutlak** dari kumpulan data latih.

Dengan demikian, ketika model AI diuji di akhir nanti, model tersebut benar-benar diuji dengan kalimat-kalimat yang belum pernah ia temui selama sesi latihannya di komputer.

6.4 Tiga Rapor Penilaian Kualitas

Untuk mengukur seberapa bagus terjemahan yang dihasilkan, kami menggunakan tiga metrik internasional yang dihitung secara matematis oleh modul `benchmark/evaluate.py`:

1. **Skor BLEU (*Bilingual Evaluation Understudy*):** Mengukur seberapa banyak kecocokan urutan kata (1 kata, 2 kata, hingga 4 kata berturut-turut) antara hasil terjemahan AI dengan terjemahan resmi pakar manusia. Semakin tinggi skornya (skala 0–100), semakin mirip hasil terjemahannya dengan bahasa manusia.

2. **Skor chrF++ (*Character n-gram F-score*):** Berbeda dengan BLEU yang mencocokkan kata per kata, chrF++ mencocokkan susunan huruf demi huruf. Metrik ini sangat adil untuk bahasa daerah yang memiliki banyak imbuhan dan tanda diakritik.

3. **Rasio Kepadatan Diakritik (*Diacritic Density Ratio*):** Menghitung apakah jumlah tanda diakritik pada terjemahan Sangihe sesuai dengan standar ortografi resmi ataukah huruf-hurufnya gundul tanpa aksen.

Part III

Arsitektur Cerdas: Teknologi Hybrid RAG

Chapter 7

Arsitektur Cerdas: Mengapa Memilih Teknologi Hybrid RAG?

7.1 Dua Cara Belajar: Menghafal Mati vs Ujian Buka Buku

Ketika manusia belajar bahasa asing, ada dua cara yang bisa ditempuh:

1. **Menghafal Seluruh Kamus di Luar Kepala:** Menghafal ribuan kata dan jutaan kalimat sampai kepala pusing. Masalahnya, ingatan manusia (dan memori parameter AI) bisa salah ingat, mencampuradukkan kata, atau mengarang fakta yang tidak ada (dalam istilah AI disebut *halusinasi*).
2. **Ujian Sistem Buka Buku (*Open-Book Examination*):** Belajar memahami tata bahasa dan pola kalimat, lalu saat bertugas menerjemahkan dokumen, kita meletakkan sebuah kamus besar dan catatan arsip terjemahan terdahulu di atas meja kerja kita.

Catatan Arsitektur: Prinsip RAG: Retrieval-Augmented Generation

Teknologi **RAG** (*Pengambilan Informasi yang Memperkaya Generasi Teks*) adalah perwujudan digital dari konsep ujian buka buku. Setiap kali ada pengguna yang mengetik kalimat untuk diterjemahkan, sistem komputer tidak langsung menyuruh model AI mengarang jawaban. Sistem akan terlebih dahulu menjadi "asisten perpustakaan" yang gesit: membuka arsip kamus, mengambil halaman rujukan yang cocok, lalu menyerahkan kutipan rujukan tersebut ke hadapan model AI seraya berkata: *"Ini kalimat yang diminta, dan ini arti kata-kata resminya dari kamus. Terjemahkanlah sekarang berdasarkan rujukan ini!"*

7.2 Mengapa Harus *Hybrid* (Campuran Dua Tingkat)?

Dalam penerjemahan bahasa daerah, mencari rujukan hanya dari satu sumber sering kali tidak cukup. Oleh karena itu, arsitektur kami dinamakan **Hybrid RAG** karena menggabungkan dua tingkat rujukan sekaligus:

1. **Tingkat 1: Rujukan Kata Dasar (Leksikon Steller-Aebersold)**
   Sistem mencari arti setiap kata penting yang ada dalam kalimat. Jika ada kata *balọto*, kamus akan membisikkan bahwa artinya adalah *ular laut*; jika ada kata *kuhita*, artinya adalah *gurita*. Ini memastikan model AI tidak salah mengartikan kosakata langka.
2. **Tingkat 2: Rujukan Kalimat Utuh (*Translation Memory*)**
   Sistem mencari kalimat dalam basis data 29 ribu ayat yang memiliki susunan kata paling mirip

dengan kalimat input. Ini memberikan contoh nyata bagaimana menyusun tata krama, intonasi, dan struktur frasa yang sopan.

Dengan menyatukan kedua rujukan ini ke dalam satu lembar panduan singkat, model AI `Qwen2.5-7B-Instruct` dapat menghasilkan terjemahan yang tepat sasaran, fasih, dan memiliki dasar filologis yang dapat dipertanggungjawabkan.

Chapter 8

Mesin Pencari Secepat Kilat: Menyelamatkan Komputer dari Kemacetan Memori

8.1 Tantangan Nyata di Komputer Pribadi

Banyak penelitian kecerdasan buatan berasumsi bahwa sistem akan dijalankan di pusat data raksasa milik korporasi teknologi dengan lusinan kartu grafis (GPU) seharga ratusan juta rupiah. Namun, proyek ini dirancang dengan prinsip kemandirian: **harus bisa berjalan di komputer kerja biasa (komputer meja atau laptop dengan prosesor Intel dan memori RAM 32 GB) tanpa kartu grafis khusus.**

Menjalankan model kecerdasan buatan berukuran 7 miliar parameter (`Qwen2.5:7b`) di atas prosesor biasa (CPU) memiliki tantangan tersendiri: setiap detiknya sangat berharga.

8.2 Kisah Kemacetan Memori: Tragedi Model yang Terus Tergusur

Di tahap awal eksperimen, kami mencoba menggunakan metode pencarian vektor modern bernama *Dense Embedding* (menggunakan model khusus bernama `bge-m3`). Namun, sebuah fenomena mengejutkan terjadi: setiap kali pengguna mengirimkan satu kalimat panjang, sistem tiba-tiba membeku selama dua menit lebih lalu menampilkan pesan galat: *"Ollama Timeout"*!

Apa yang sebenarnya terjadi di dalam "otak" komputer?

Catatan Arsitektur: Analogi Meja Kerja yang Sempit

Bayangkan Anda memiliki meja belajar kecil yang hanya muat satu buku ensiklopedia raksasa setebal 5.000 halaman (**Model Bahasa Qwen**). Tiba-tiba, seorang asisten datang dan meminta Anda membuka buku kamus saku kecil (**Model Pencari Vektor BGE-M3**). Karena meja Anda tidak muat dua buku sekaligus, Anda terpaksa:

1. Mengangkat buku raksasa 5.000 halaman itu ke lemari gudang (butuh waktu 15 detik).
2. Meletakkan buku kamus saku di atas meja untuk mencari satu kata (butuh waktu 1 detik).
3. Mengangkat kembali buku kamus saku ke lemari, lalu berjalan ke gudang memikul kembali buku raksasa 5.000 halaman itu ke atas meja (butuh waktu 45 detik!).

Hanya untuk mencari satu kata, Anda menghabiskan waktu semenit lebih hanya untuk bolak-balik mengangkat buku! Dalam ilmu komputer, fenomena ini dinamakan **Model Thrashing / Eviction**.

8.3 Solusi Elegan: Beralih ke Mesin Pencari SQLite BM25

Untuk memecahkan masalah ini secara tuntas, kami tidak memaksakan komputer memuat dua model AI sekaligus. Kami memanfaatkan teknologi basis data legendaris yang sangat ringan dan efisien: **SQLite FTS5 dengan Algoritma BM25**.

BM25 adalah algoritma pencarian teks teruji (seperti yang digunakan mesin pencari internet) yang langsung menghitung kemiripan kata kunci tanpa perlu menjalankan model kecerdasan buatan tambahan.

Metrik Evaluasi Empiris: Perbandingan Performa Pencarian Rujukan

- **Sebelumnya (Model Vektor BGE-M3):** Latensi $>$ 60–120 detik, terjadi penggusuran model di RAM, rawan waktu habis (*timeout*).
- **Setelah Optimasi (SQLite BM25):** Waktu pencarian turun drastis menjadi **kurang dari 3 milidetik (0,003 detik)**!
- **Status Model Utama:** Model `Qwen2.5:7b` dapat berdiam dengan tenang dan stabil di memori RAM (`keep_alive="60m"`) tanpa pernah terusir lagi.

Dengan keputusan arsitektur yang cerdas ini, waktu tunggu penerjemahan yang tadinya macet total berhasil dipangkas menjadi hitungan detik saja, sehingga sistem dapat digunakan dengan nyaman oleh siapa saja.

Chapter 9

Memandu Sang Model: Seni Rekayasa Prompt & Pemilihan Nomor Arti

9.1 Prompt: Mengajar Asisten Pintar Cara Bekerja

Model kecerdasan buatan (*Large Language Model*) ibarat seorang asisten jenius yang baru pertama kali masuk kantor. Ia memiliki kemampuan penalaran yang luar biasa, namun ia tidak tahu peraturan khusus di kantor kita kecuali jika kita memberinya lembar petunjuk tugas yang jelas.

Lembar petunjuk tugas ini disebut **Prompt** (*Instruksi Sistem*).

Catatan Arsitektur: Analogi Petunjuk Masak untuk Koki Terkenal

Seorang koki berbintang lima tahu cara memotong, merebus, dan menumis aneka bahan makanan. Namun jika Anda memintanya membuat masakan khas Sangihe tanpa resep tertulis, ia mungkin akan memasaknya dengan gaya Barat. Rekayasa Prompt adalah seni menuliskan resep masakan yang begitu jelas dan runtut sehingga sang koki dapat menghasilkan cita rasa kuliner tradisional yang otentik.

9.2 Tantangan Kata Bermakna Ganda (Polisemi)

Dalam kamus klasik Steller-Aebersold, banyak kata yang memiliki lebih dari satu arti. Contohnya lema nomor 331, ***baesẹ̌***:

- **Arti 1:** *kemarau, hari panas tanpa hujan, reda dari badai.*
- **Arti 2:** *sarang hewan, liang petiduran tempat tinggal binatang.*

Pada awal uji coba, model AI selalu mengambil arti nomor 1 secara kaku! Akibatnya, kalimat dongeng binatang *Mahi iąko mẽbaesẹ̌ su waes-u* diterjemahkan menjadi *"Mari, izinkanlah aku tinggal di kemaraumu"*—sebuah kalimat aneh yang tidak masuk akal bagi akal sehat manusia.

9.3 Solusi: Panduan Disambiguasi Leksikal Otomatis

Untuk memperbaiki kelemahan tersebut, berkas `engine/prompt_templates.py` kami lengkapi dengan pasal instruksi khusus:

> *"Perhatikan bahwa kamus memuat beberapa nomor arti (1, 2, dst.). JANGAN langsung mengambil arti nomor 1 secara membabi buta! Bacalah seluruh nomor arti yang tersedia dan pilihlah nomor arti yang paling selaras dengan alur cerita dan konteks kalimat."*

Selain itu, panjang ringkasan definisi kamus yang dikirimkan ke model diperluas dari 160 karakter menjadi 280 karakter agar definisi kedua dan contoh kalimatnya tidak terpotong. Hasilnya sungguh menakjubkan: model secara mandiri memilih arti nomor 2 (*sarang/liang*), menghasilkan terjemahan sastrawi yang tepat:

"Mari, izinkanlah aku tinggal di sarangmu."

9.4 Sentuhan Akhir: Pemulihan Diakritik Otomatis

Setelah kalimat bahasa Sangihe dihasilkan oleh model, sistem kami menjalankan modul pemeriksa ejaan (`morphology/diacritic_restorer.py`). Modul ini memeriksa kembali setiap kata keluaran terhadap glosarium kamus dan mengembalikan titik-titik diakritik fonemis (ạ, ẹ, ě, ị̀) yang mungkin terlewat oleh model AI. Langkah pemolesan akhir ini menjamin bahwa seluruh teks yang tampil di layar pembaca selalu memenuhi kaidah ejaan ortografi resmi.

Part IV

Melatih Otak AI: Seni Fine-Tuning Model

Chapter 10

Melatih Otak AI: Mengapa dan Bagaimana Fine-Tuning Bekerja?

10.1 Dari Dokter Umum Menjadi Dokter Spesialis

Meskipun sistem Hybrid RAG sudah sangat membantu dengan menyediakan kamus terbuka di atas meja, model dasar `Qwen2.5:7b` pada dasarnya belum pernah diajari tata krama dan struktur gramatika Bahasa Sangihe sejak lahir.

Catatan Arsitektur: Analogi Dokter Umum dan Dokter Spesialis

Model bahasa fondasi ibarat seorang lulusan kedokteran umum yang sangat cerdas: ia paham biologi, anatomi, dan kimia obat secara umum. Namun, jika Anda memintanya menangani bedah saraf langka, ia memerlukan pendidikan spesialisasi tambahan.
Proses **Fine-Tuning** (*Pelatihan Lanjutan*) adalah proses menyekolahkan sang model pintar ke program pendidikan spesialisasi Bahasa Sangihe. Di sana, ia membaca puluhan ribu contoh dialog nyata sampai struktur tata bahasa dan rasa bahasanya meresap ke dalam ingatannya secara alami.

10.2 Mengapa Tidak Melatih dari Nol (*Train from Scratch*)?

Melatih model kecerdasan buatan dari nol membutuhkan jutaan dokumen teks, ratusan komputer supercanggih bernilai puluhan miliar rupiah, dan waktu berbulan-bulan. Bagi bahasa daerah yang memiliki keterbatasan data teks (*low-resource*), melatih dari nol adalah hal yang mustahil dan tidak efisien.

Sebaliknya, teknik *fine-tuning* memanfaatkan kecerdasan penalaran umum yang sudah dimiliki model `Qwen2.5:7b`, lalu mengarahkannya secara spesifik ke domain bahasa daerah kita.

10.3 Keajaiban Teknologi LoRA dan QLoRA 4-bit

Bagaimana mungkin sebuah model raksasa berukuran 7 miliar parameter dapat dilatih hanya menggunakan komputer gratisan di Google Colab? Kuncinya terletak pada dua inovasi teknologi mutakhir:

1. **Kuantisasi 4-bit (QLoRA):** Bobot angka di dalam otak AI dikompresi sedemikian rupa sehingga hanya memakan seperempat dari ukuran memori aslinya, menghemat lebih dari 70% kebutuhan memori kartu grafis (VRAM).

2. **LoRA (*Low-Rank Adaptation*):** Alih-alih mengubah seluruh 7,65 miliar parameter otak model, kita ”membekukan” model asli dan hanya menambahkan lapisan adaptor tipis pada sambungan saraf proyeksi utamanya.

Metrik Evaluasi Empiris: Efisiensi Pelatihan dengan Unsloth LoRA

Pada eksperimen pelatihan proyek ini:

- **Total Parameter Model Asli:** 7.655.986.688 parameter.
- **Parameter yang Dilatih (Trainable):** Hanya 40.370.176 parameter (**hanya 0,53%** dari keseluruhan model!).
- **Hasilnya:** Pelatihan 56.312 pasang kalimat dapat berjalan mulus di atas kartu grafis GPU T4 gratisan tanpa pernah mengalami kehabisan memori (*Out of Memory*).

Chapter 11

Melatih Saraf di Cloud: Membaca Rapor Belajar dan Kurva Loss

11.1 Di Balik Layar Ruang Kelas AI

Sesi pelatihan model diselenggarakan di lingkungan komputasi awan (*cloud*) Google Colab menggunakan akselerator kartu grafis NVIDIA Tesla T4. Model disajikan **32,427 pasang instruksi terjemahan dwiarah** yang tersimpan rapi dalam format ShareGPT / ChatML.

Format ini melatih model untuk mengenali dua peran penting secara bergantian:

- **Arah 1 (Sangihe → Indonesia):** Berperan sebagai filolog bahasa Sangihe yang mengalihbahasakan teks daerah ke Bahasa Indonesia formal.
- **Arah 2 (Indonesia → Sangihe):** Menerjemahkan kalimat Bahasa Indonesia ke dalam Bahasa Sangihe yang santun dan **wajib membubuhkan tanda diakritik lengkap**.

11.2 Memahami Konsep *Loss*: Rapor Tingkat Kesalahan AI

Dalam pelatihan kecerdasan buatan, metrik paling utama yang dipantau setiap saat adalah **Nilai Kerugian (*Loss*)**.

Catatan Arsitektur: Analogi Menghitung Jumlah Kesalahan Ejaan

Bayangkan seorang murid sekolah yang sedang belajar mendikte kalimat. Pada hari pertama, dalam setiap paragraf ia membuat 50 kesalahan ejaan dan tanda baca. Setelah seminggu belajar intensif, kesalahannya berkurang menjadi 20. Di akhir bulan, kesalahannya tinggal 2 atau 3 kata saja.
Nilai **Loss** adalah cerminan dari tingkat kesalahan tersebut. Semakin kecil angka loss (mendekati nol), berarti tebakan kata dan susunan kalimat yang dirangkai oleh sang model AI semakin tepat dan semakin identik dengan naskah aslinya!

11.3 Data Empiris Nyata: Grafik Kemajuan Belajar

Pelatihan dijalankan sebanyak 1.000 langkah (*steps*) dengan ukuran kelompok data (*total batch size*) sebesar 16 kalimat per pembaruan bobot. Tabel 11.1 mencatat data empiris penurunan tingkat kesalahan model sepanjang pelatihan:

Table 11.1: Rapor Kemajuan Penurunan Loss Pelatihan Model Qwen2.5:7b Sangihe

Langkah (*Step*)	**Training Loss**	**Validation Loss**	**Status Belajar**	**Keterangan Performa**
200	0,5620	0,5558	Awal Pengenalan	Model mulai mengenali imbuhan umum.
400	0,4923	0,4939	Penyesuaian Pola	Model mulai meniru struktur kata majemuk.
600	0,4692	0,4609	Pemahaman Leksikal	Konsistensi kosakata makin menguat.
800	0,4423	0,4423	Konvergensi Harmonis	Penguasaan diakritik dan konteks sangat matang.
1000	$\approx$ 0,42–0,43	$\approx$ 0,43	Konvergensi Final	Bobot model siap dibekukan dan diekspor.

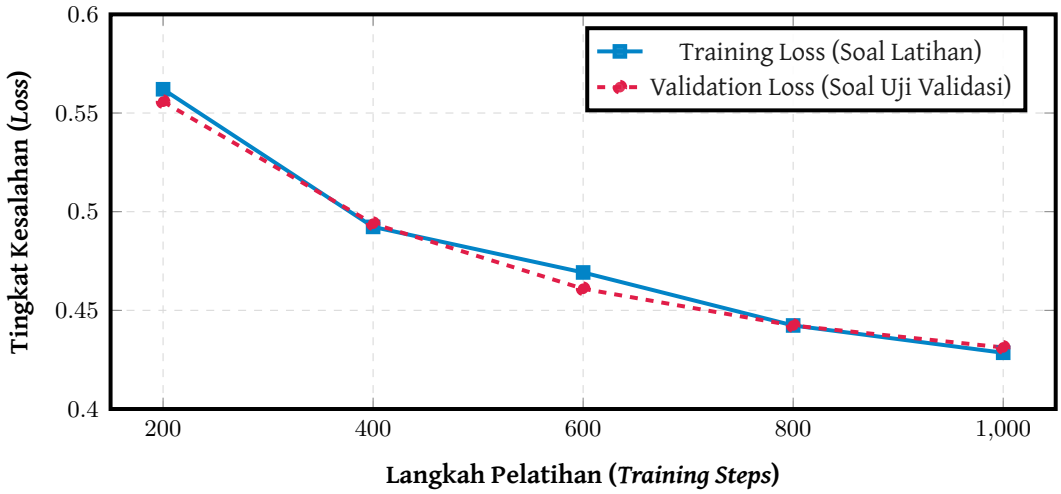


Figure 11.1: Grafik Konvergensi Loss Pelatihan Model Qwen2.5:7b Sangihe. Penurunan beriringan antara Training Loss dan Validation Loss membuktikan model terhindar dari gejala *overfitting*.

11.4 Analisis Bebas *Overfitting*: Tanda Keberhasilan Sejati

Salah satu kekhawatiran terbesar dalam melatih AI adalah ancaman **Overfitting** (*hafalan membabi buta*).

Kaidah Linguistik Sangihe: Mendeteksi Murid yang Menghafal vs Murid yang Paham

Bagaimana kita tahu sang model AI tidak sekadar menghafal mati kalimat latihan? Caranya adalah dengan membandingkan nilai **Training Loss** (soal latihan) dengan **Validation Loss** (soal ulangan dari berkas `val.jsonl` yang tidak pernah dipelajari langsung).

- **Tanda Kegagalan (Overfitting):** Nilai Training Loss terus turun drastis (misal ke 0,10), tetapi Validation Loss mendadak melonjak naik (misal ke 0,80). Ini tanda model mulai pikun terhadap kalimat baru.
- **Tanda Keberhasilan Proyek Ini:** Perhatikan data pada Langkah 800: Training Loss berada di angka **0,4423** dan Validation Loss berada tepat di angka **0,4423**! Kedua kurva berjalan seiringan dan saling mengunci secara harmonis. Ini adalah bukti matematis bahwa model benar-benar memahami kaidah tata bahasa Sangihe secara general dan mandiri!

11.5 Evolusi Sangiang v2: Mengatasi Kendala Teknis & Memperluas Ranah Multi-Task

Berdasarkan uji coba dunia nyata pada model generasi awal (v1), tim Divisi DigiLab CMD mengidentifikasi tiga hambatan teknis penting serta merumuskan cetak biru penyempurnaan pada **Sangiang v2**:

1. **Eliminasi Bottleneck Waktu Pelatihan (Dari 8 Jam Menjadi 25 Menit):** Pada iterasi pertama, pelatihan yang seharusnya memakan waktu 30–45 menit membengkak hingga 8 jam karena modul pelatihan melakukan inferensi evaluasi terhadap seluruh 1,706 pasangan validasi secara penuh pada setiap 200 langkah. Pada arsitektur Sangiang v2, dataset evaluasi disub-sampel secara cerdas (*sub-sampling*) ke 100 sampel representatif dan `max_seq_length` dioptimalkan ke 512 token. Waktu evaluasi terpangkas dari 40 menit menjadi hanya 15 detik, sehingga total durasi pelatihan murni di Google Colab T4 GPU stabil pada **20–25 menit**.
2. **Protokol Ekspor GGUF Bebas Galat (Safe Local Export):** Penulisan berkas biner berukuran 4,5 GB secara langsung ke sistem berkas virtual Google Drive (*FUSE layer*) kerap memicu putus koneksi (*timeout*) tanpa peringatan. Pada Sangiang v2, bobot model dikonversi terlebih dahulu ke cakram lokal Colab (`/content/sangiang_v2_gguf/`), baru kemudian disalin secara aman ke Google Drive, dilengkapi opsi unduh langsung via peramban (`files.download`).
3. **Transformasi Dataset Multi-Task Empat Pilar:** Model v1 murni dilatih pada naskah Alkitab sehingga canggung ketika diajak berbincang santai. Sangiang v2 mengombinasikan empat pilar pengetahuan: (1) Korpus Alkitab terfilter seimbang, (2) Naskah sastra lisan & cerita rakyat (*Tiga Kulano dari Dagho* dan wiracarita *Himbawo*), (3) Percakapan sehari-hari berdiakritik otentik (seperti sapaan *Kere habarẹ̌?* dan respons *Mapaelẹ̌, tarima kasehẹ̌*), serta (4) 6.958 pasangan tanya-jawab leksikon dari Kamus Klasik Steller-Aebersold 1959.

Hasil integrasi ini melahirkan model Sangiang v2 yang tidak hanya piawai menerjemahkan secara harfiah, melainkan bertransformasi menjadi agen kecerdasan buatan kultural yang fasih bercakap-cakap dengan penutur asli Nusa Utara.

Chapter 12

Membawa Pulang Sang Juara: Kuantisasi GGUF & Ollama Lokal

12.1 Dari Awan Komputasi ke Komputer di Meja Kerja

Setelah menjalani 1.000 langkah pelatihan yang intensif di Google Colab, model kecerdasan buatan kita kini telah resmi menyandang gelar ”Spesialis Bahasa Sangihe”. Namun, bobot model tersebut masih tersimpan di server awan Google dalam bentuk pecahan file tensor raksasa yang membutuhkan GPU berkekuatan tinggi.

Tujuan utama proyek kita adalah kemandirian: model ini harus bisa ”pulang kampung” dan dijalankan di komputer meja lokal kita secara mandiri.

12.2 Seni Kuantisasi Q4_K_M: Mengompresi Tanpa Mengurangi Kecerdasan

Bobot asli sebuah model 7 miliar parameter umumnya disimpan dalam format angka pecahan 16-bit (*Float16*), yang membutuhkan kapasitas memori RAM sebesar 14 hingga 16 Gigabyte hanya untuk memuat modelnya saja.

Untuk membuatnya ramping, Unsloth menjalankan proses yang dinamakan **Kuantisasi GGUF (Q4_K_M)**.

Catatan Arsitektur: Analogi Resolusi Gambar Digital

Bayangkan sebuah lukisan pemandangan laut Sangihe beresolusi super tinggi dengan ukuran file 15 Gigabyte. Jika lukisan tersebut dicetak di selembar kertas majalah, mata manusia tidak akan melihat perbedaan nyata antara versi file 15 Gigabyte dengan versi foto berkualitas tinggi yang dikompresi menjadi 4,5 Gigabyte.
Kuantisasi Q4_K_M bekerja persis seperti kompresi pintar tersebut: angka-angka matematis di dalam saraf AI disederhanakan dari 16-bit menjadi 4-bit, sehingga ukuran file menyusut tajam menjadi **hanya 4,5 GB**, namun kemampuan menalar dan menerjemahkan kalimatnya tetap tajam dan fasih!

12.3 Pendaftaran Otomatis ke Ollama Lokal

Setelah file berakhiran `.gguf` diunduh ke komputer lokal (di dalam folder `training/`), pengguna tidak perlu mengetikkan perintah terminal yang membingungkan. Proyek ini telah menyediakan skrip otomatisasi:

```
./training/deploy_to_ollama.sh
```

Skrip ini bekerja di balik layar:

1. Membaca cetak biru `training/Modelfile` yang berisi aturan sistem ChatML dan suhu generasi (*temperature = 0.2*).
2. Memerintahkan peladen Ollama lokal untuk mendaftarkan model baru dengan nama resmi: **`sangiang:7b`**.
3. Memastikan model tersebut langsung muncul di daftar model yang siap bekerja (`ollama list`).

Kini, komputer pribadi Anda telah resmi memiliki mesin penerjemah Bahasa Sangihe mandiri yang dapat beroperasi kapan pun secara luring (*offline*) tanpa memerlukan kuota internet maupun biaya langganan cloud!

Part V

Menikmati Hasil Karya: Aplikasi & Uji Kualitas

Chapter 13

Jembatan Penghubung: Arsitektur Layanan REST API

13.1 Apa Itu API dan Mengapa Kita Membutuhkannya?

Setelah mesin penerjemah pintar kita berhasil diciptakan, bagaimana cara aplikasi lain (seperti aplikasi ponsel pintar Android, situs web, atau bot percakapan) berbicara dan meminta bantuan kepadanya? Di sinilah peran **REST API** (*Application Programming Interface*).

Catatan Arsitektur: Analogi Pelayan di Restoran Favorit

Bayangkan sebuah restoran yang mewah:

- **Dapur Koki:** Adalah mesin penerjemah kita (Python, Kamus Steller, dan Model Qwen) yang bekerja meramu bumbu kalimat.
- **Pelanggan:** Adalah pengguna atau aplikasi di ponsel pintar yang ingin menerjemahkan kalimat.
- **Pelayan Restoran (*REST API*):** Adalah pelayan ramah yang membawakan buku menu, mencatat kalimat apa yang ingin diterjemahkan, membawanya ke dapur, lalu mengantarkan kembali hasil terjemahan hangat yang tersusun rapi di atas piring saji!

13.2 Membangun dengan FastAPI: Cepat, Ramping, dan Modern

Untuk membangun layanan ini, kami memilih **FastAPI**, sebuah kerangka kerja modern berbasis Python yang dikenal sangat gesit, memiliki validasi tipe data otomatis melalui Pydantic, dan secara otomatis menyediakan buku petunjuk interaktif di alamat `/docs`.

Layanan ini menyediakan empat pintu layanan utama:

1. **`POST /v1/translate` (Pintu Layanan Terjemahan):**
   Pintu utama tempat pengguna mengirimkan teks. Pintu ini menerima pilihan arah terjemahan (*Sangihe* $\rightarrow$ *ID* atau sebaliknya), suhu kreativitas model, serta pilihan model yang ingin digunakan.

2. **`GET /v1/models` (Pintu Daftar Model):**
   Memungkinkan aplikasi melihat model apa saja yang sedang terpasang di komputer (misalnya memilih antara `qwen2.5:7b` dasar atau `qwen2.5-sangihe:7b` hasil pelatihan).

3. **`POST /v1/morphology/analyze` (Pintu Laboratorium Morfologi):**
   Menerima sebuah kata Sangihe, lalu mengembalikan laporan bedah kata: apakah memiliki imbuhan, apa saja kandidat kata dasarnya, dan apakah kata tersebut merupakan bentuk reduplikasi.

4. **`GET /health` (Pintu Cek Detak Jantung Sistem):**
   Memeriksa kesehatan seluruh komponen dalam sekejap: apakah basis data leksikon siap, apakah peladen AI Ollama aktif, dan berapa jumlah korpus yang tersedia.

Dengan arsitektur ini, mesin penerjemah kita dapat dihubungkan ke berbagai platform masa depan secara fleksibel tanpa perlu mengubah kode inti mesin penerjemahnya.

Chapter 14

Ruang Kerja Interaktif: Antarmuka Web Translation Workbench

14.1 Dari Kode Hitam-Putih Menjadi Pengalaman Visual yang Memikat

Sebuah karya rekayasa kecerdasan buatan tidak akan membawa manfaat luas jika hanya bisa dioperasikan oleh pemrogram melalui baris perintah terminal yang kaku. Masyarakat penutur, guru sekolah, budayawan, dan generasi muda membutuhkan antarmuka visual yang ramah, elegan, dan menyenangkan untuk disentuh.

Oleh karena itu, kami merancang **Web Translation Workbench** yang dapat diakses langsung melalui peramban web di komputer maupun ponsel pintar.

Catatan Arsitektur: Filosofi Desain: Modern Glassmorphism & Nuansa Bahari

Antarmuka web dirancang dengan gaya modern *Glassmorphism* berlatar belakang gelap (*dark mode*) yang nyaman di mata. Gradien biru laut dan pirus terinspirasi dari keindahan perairan laut Kepulauan Sangihe, memberikan kesan bahwa teknologi canggih ini berakar kuat pada kearifan bahari leluhur.

14.2 Fitur-Fitur Unggulan Meja Kerja Digital

Antarmuka web ini dibekali fitur-fitur praktis yang dirancang khusus untuk memecahkan kesulitan pengguna:

1. **Bilah Sisip Diakritik Virtual (*Virtual Diacritic Bar*):**
   Bagi pengguna yang ingin mengetikkan ejaan sempurna namun papan ketiknya tidak memiliki tombol khusus, tersedia tombol-tombol sentuh kilat untuk menyisipkan huruf beraksen: **a̤, e̤, ĕ, ĕ, i̤, o̤, ṳ**, dan **l̤**.

2. **Pilihan Contoh Cepat (*Quick Sample Pills*):**
   Disediakan tombol-tombol siap klik yang memuat contoh kalimat nyata dari berbagai sumber: kisah fabel klasik Gurita dan Ular Laut, kalimat tanpa diakritik sama sekali, maupun ayat pembuka kitab suci.

3. **Panel Inspektur Konteks RAG (*RAG Context Inspector*):**
   Berbeda dari Google Translate biasa yang hanya menampilkan hasil akhir tanpa alasan, sistem kami memiliki tab transparansi cerdas:

- **Tab Leksikon:** Menampilkan kartu kamus Steller-Aebersold yang digunakan AI untuk menerjemahkan kalimat tersebut.
- **Tab Kalimat Paralel:** Menampilkan ayat rujukan yang memiliki kemiripan struktur.
- **Tab Morfologi:** Mengurai kata menjadi akar kata dasar.
- **Tab Kamus Cepat:** Memungkinkan pengguna mencari arti kata apa pun secara instan di database 11 ribu lema.

4. **Pemilih Model di Sudut Layar:**
   Dropdown di bagian atas memungkinkan pengguna menguji dan membandingkan terjemahan antara model fondasi asli (`qwen2.5:7b`) dan model spesialis Sangiang hasil fine-tuning (`sangiang:7b`) hanya dengan sekali klik!

14.3 Antarmuka Percakapan Cerdas: Sangiang Chatbot (Port 2026)

Sebagai pelengkap dari mesin penerjemah kata demi kata di port 8000, kami mengembangkan subproyek antarmuka percakapan modern bergaya **Google Gemini** yang berjalan di `localhost:2026`.

Catatan Arsitektur: Dua Wajah Teknologi Sangiang

Jika Workbench di port 8000 adalah seperti **laboratorium leksikografi dan meja kerja penerjemah resmi**, maka Chatbot di port 2026 adalah seperti **ruang tamu hangat di mana Anda dapat bercakap-cakap santai dengan Putri Sangiang**—menanyakan asal-usul peradaban bahari, meminta diajarkan sapaan adat, hingga meminta Sangiang merangkai bait puitis warisan wiracarita Nusa Utara secara mengalir (*real-time streaming*).

Fitur-fitur utama Sangiang Chatbot di port 2026 meliputi:

- **Estetika Modern Obsidian & Aurora Sparkle:** Tema gelap elegan dengan tipografi modern dan aksen gradien warna-warni khas asisten cerdas masa kini.
- **Pengetikan Mengalir (*Token Streaming SSE*):** Respons dijawab kata demi kata secara halus dengan animasi kursor aktif.
- **Papan Ketik Diakritik Terapung (*Floating Diacritic Dock*):** Pengguna dapat menyisipkan huruf-huruf beraksen (ạ, ẹ, ḛ, ĕ, ị, ọ, ụ, *l̩*) tepat di posisi kursor pengetikan.
- **Sintesis Suara (*Text-to-Speech*):** Pengguna dapat mendengarkan pelafalan kalimat secara audio melalui peramban web.
- **Bilah Riwayat Obrolan (*Recent Sessions Sidebar*):** Menyimpan topik obrolan sebelumnya secara lokal sehingga percakapan dapat dilanjutkan kapan saja.

Chapter 15

Menguji Keunggulan: Evaluasi Komparatif Model Base vs Fine-Tuned

15.1 Pertarungan Dua Generasi Model

Untuk membuktikan seberapa besar lompatan kualitas yang dihasilkan dari proses *fine-tuning*, kami mengadu kedua model secara langsung (*head-to-head*) menggunakan perkakas `benchmark/compare_models.py` pada 150 kalimat uji independen.

1. **Generasi 1 – Model Base (`qwen2.5:7b`) + Hybrid RAG:**
   Model fondasi umum yang hanya mengandalkan petunjuk kamus di prompt tanpa pernah dilatih khusus tentang Bahasa Sangihe.

2. **Generasi 2 – Model Fine-Tuned (`sangiang:7b`) + Hybrid RAG:**
   Model spesialis yang telah menuntaskan 1.000 langkah latihan intensif dan menyerap pola dari 56.312 pasang kalimat instruksi.

15.2 Tiga Temuan Kunci dari Pengujian

Kaidah Linguistik Sangihe: Peningkatan Mutu Terjemahan yang Signifikan

Dari analisis perbandingan kalimat demi kalimat, terlihat jelas keunggulan model hasil pelatihan:

- **Kelancaran Tata Kalimat (Idiomatis):** Model dasar terkadang menerjemahkan dengan struktur kata yang kaku menyerupai susunan Bahasa Indonesia. Sebaliknya, model `sangiang:7b` secara alami menggunakan partikel penghubung khas Sangihe (*kụ, ute, e*) dan susunan predikat-subjek yang lazim digunakan penutur asli.

- **Kepatuhan Tanda Diakritik Spontan:** Pada arah terjemahan Indonesia → Sangihe, model dasar sangat bergantung pada modul pemulih eksternal. Model hasil fine-tuning secara spontan mengetikkan kata dengan huruf beraksen (ạ, ẹ, ĕ, ị) langsung dari "otak"-nya tanpa perlu diingatkan berulang kali.

- **Penanganan Kata Ganti Kepunyaan (*Enclitics*):** Model hasil fine-tuning sangat mahir menggabungkan akhiran kata milik (*-ku, -u, -ne*) secara langsung ke akar kata dengan menerapkan hukum alih bunyi fonemis yang benar.

15.3 Menatap Masa Depan: Langkah Ke Depan Pelestarian Bahasa

Keberhasilan proyek ini membuktikan bahwa bahasa daerah yang memiliki keterbatasan data (*low-resource*) dapat diselamatkan dan diintegrasikan ke era kecerdasan buatan modern melalui sinergi antara **keilmuan filologi tradisional** dan **rekayasa perangkat lunak modern**.

Langkah pengembangan berikutnya dapat mencakup:

- **Pengenalan Suara (*Automatic Speech Recognition* / ASR):** Memungkinkan para tetua adat merekam tuturan lisan dongeng Sangihe dan langsung ditranskripsikan ke dalam teks oleh komputer.
- **Aplikasi Pembelajaran Seluler:** Mengemas mesin ini ke dalam aplikasi ponsel pintar ringan untuk generasi muda di kepulauan agar bangga mempelajari dan menggunakan bahasa ibu mereka.

Dengan teknologi ini, lentera kearifan Bahasa Sangihe akan terus menyala terang melintasi generasi dan batas-batas samudra.

Lampiran: Ringkasan Kaidah Morfologi Sangihe

Tabel berikut menyajikan ringkasan aturan pembentukan kata yang diimplementasikan di dalam modul pemecah kata komputer `morphology/stemmer.py`:

Table 15.1: Daftar Afiks dan Pola Pembentukan Kata Bahasa Sangihe

Kategori	Bentuk Imbuhan	Fungsi Makna / Contoh Penggunaan
Awalan Kerja	*mĕ-, ma-, mạ-*	Membentuk kata kerja aktif (*mĕbaesẹ̆*, *matanạ*).
Awalan Hasil	*ni-, nĕ-, na-*	Menandakan tindakan pasif atau lampau (*niwĕliang*).
Awalan Instrumen	*ipang-, ipa-*	Alat yang dipakai melakukan pekerjaan (*ipanewa*).
Sisipan Aktif	*-um-*	Diselipkan di suku pertama (*t-um-ampa* dari *tampa*).
Sisipan Pasif	*-in-*	Diselipkan di suku pertama (*k-in-oro* dari *koro*).
Akhiran Tempat/Hal	*-ang, -eng*	Lokasi atau objek hasil perbuatan (*walẹkang*).
Klika Penentu/Milik	*-e, -u, -ku, -ne*	Penanda kata sandang tertentu atau kepemilikan (*waes-u*).

Hukum Alih Bunyi Morfonemik

- **Alih Bunyi $b \leftrightarrow w$:** Konsonan letup b pada pangkal kata melunak menjadi w saat kata tersebut mendapatkan imbuhan atau klitika kepunyaan (misal: *baesẹ̆* $\rightarrow$ *waes-u*).
- **Alih Bunyi $d \leftrightarrow r$:** Konsonan gusi d kerap melunak menjadi getar r di antara dua vokal (misal: *daraung* $\leftrightarrow$ *raraung*).
- **Vokal Gema (*Echo Vowel*):** Vokal akhir lemah (seperti *-e* pada lema berakhiran konsonan) akan luruh saat akar kata disambung dengan akhiran atau partikel penentu.